A 10-iteration bounded check misses 1.3% of wrong loop invariants

CSE 216, Team 10. Preliminary results, version 2

Working notes, 11 November 2026

AI-drafted notes. Course policy requires the report text to be written by the team. Every number comes from `src/analyze.py` in `https://github.com/arelyx/invariant-grader-gap`.

Summary

We graded 3,935 model-written loop invariants with a bounded model checker (ESBMC) and a full verifier (UAutomizer). The Loop-R1 paper trains models with a bounded check at a bound under 10, says it may accept false invariants, and gives no count.

27 of 2,016	wrong invariants accepted at a bound of 10 (1.3%, 95% interval 0.9–1.9)
4 of those 27	are caused by the bound; the other 23 are a pointer-modelling difference
34	invariants UAutomizer proved that are false when an `int` overflows

1 Finding 1 (most important): the tools disagree about C more than about the bound

We expected one kind of disagreement: the bounded check accepts something that fails after iteration 10. We found three, and that one is the smallest.

Kind	Candidates	Example
Fails after the bound	4	`st == 0` on `nested_delay_notd2`. Accepted up to bound 20, rejected at 50.
Pointer compared with zero	23	`0 < head` on six linked-list programs. UAutomizer refutes it. ESBMC accepts it at every bound it finishes, including 50. Cause not yet traced.
Signed overflow	34	Below.

The overflow case, from `benchmark46_disjunctive`:

```
int x, y, z;                          // arbitrary, at least one positive
while (nondet()) {
  assert(x > 0 || y > 0 || z > 0);    // candidate invariant
  if (x > 0) x++;
  if (y > 0) y++; else z++;
}
```

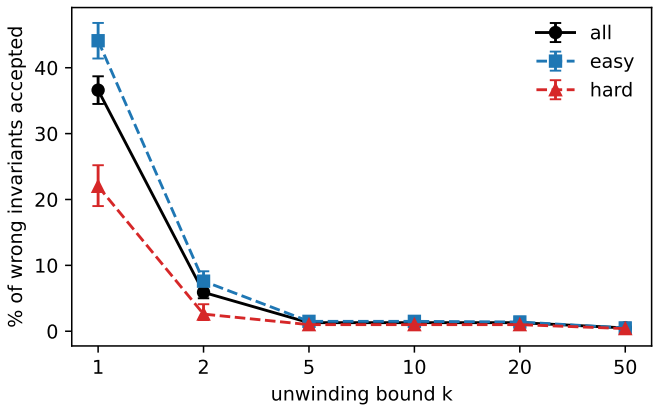
UAutomizer proves it. ESBMC finds a failure on the second iteration: `x = -1, y = -1, z = 2147483647`, then `z++` wraps to -2147483648. ESBMC treats `int` as 32 bits. UAutomizer does not consider the wrapping run. With unbounded integers (`--ir`) ESBMC agreed with UAutomizer on all three cases we re-ran.

Consequence for training: a model rewarded by UAutomizer learns invariants that break on 32-bit hardware, and a model rewarded by ESBMC is penalised for invariants that hold over the mathematical integers.

2 Finding 2: the bound matters below 5 and not above

bound	refuted invariants accepted (of 2,016)	true invariants timed out (of 1,628)
1	737 (36.6%)	1.8%
2	119 (5.9%)	2.8%
5	27 (1.3%)	9.0%
10	27 (1.3%)	12.8%
20	26 (1.3%)	24.6%
50	10 (0.5%)	41.7%

Wrong invariants from these models usually fail on iteration 1 or 2. Going from bound 10 to 50 catches 17 more and triples the timeouts.



3 Finding 3: the bounded check is a cheap way to choose among samples

Eight samples per loop from Wonda's fine-tuned 4B model, 362 loops. Median time per check: ESBMC 0.4 s, UAutomizer 7.5 s.

Which sample is kept	Proved by UAutomizer
The first one	168/362 (46%)
The first one ESBMC accepts at bound 10	253/362 (70%)
Upper limit: any of the eight is provable	277/362 (77%)

4 What could overturn this

1. **Short loops.** 103 of the 362 loops stop after a fixed small count, 26 of them after one iteration. A bounded check cannot miss anything there. On long-running loops the 1.3% could be much higher. This is the first thing to test.
2. **We guessed Loop-R1's setup.** Their code is unpublished. We used `esbmc --unwind k --no-unwinding-assertions` from one sentence in the paper. Their reward also has a Z3 check that we did not build.
3. **291 candidates have no verdict.** UAutomizer timed out. ESBMC accepted 109 of them at bound 10, and any of those could be wrong.
4. **One model family.** All four models are Qwen3.
5. **Thin prior-work search.** We found no earlier count. We did not search exhaustively.

5 Setup

Programs	InvBench as shipped with Wonda: 219 C programs, 362 loops
Candidates	1,086 from untrained Qwen3 0.6B, 1.7B and 4B, one per loop each. 2,896 from Wonda's fine-tuned 4B, eight per loop. 3,935 parse as C.
What is checked	The program with `assert(candidate)` at the top of the loop body
Full verifier	UAutomizer, SV-COMP 2025. Limit 30 s, or 120 s on the 123 slow loops. Result: 1,628 proved, 2,016 refuted, 291 no answer.
Bounded check	ESBMC 8.6 with `--32 --force-malloc-success` and pointer, bounds and division checks off. Bounds 1, 2, 5, 10, 20, 50. Limit 30 s.

Two flags changed the numbers. Without `--force-malloc-success` ESBMC lets `malloc` return `NULL` and rejects `x != NULL`, which UAutomizer proves. The benchmark also uses `NULL` without a header, which ESBMC needs defined.

6 Next three experiments

1. Remove the iteration caps from the 103 capped programs and re-measure.
2. Run ESBMC with unbounded integers on everything, so overflow disagreements can be subtracted out.
3. Find out why the tools disagree on `0 < head`.

Sources

Loop-R1 preprint, `https://www.ai2wj.com/data/Loop-R1-preprint.pdf`. Wonda, arXiv:2603.15510. Quokka / InvBench, arXiv:2509.21629. ESBMC 8.6, `https://github.com/esbmc/esbmc`. Ultimate Automizer, SV-COMP 2025 release.